Complete Technical Documentation: GSAP, Three.js & WebGL Shaders

Purpose: Exhaustive reference documentation for building premium web experiences with 3D card flips and sophisticated animated backgrounds. Optimized for AI coding assistant (Claude Code) quick lookup and implementation.

PART 1: GSAP ANIMATION LIBRARY

1.1 Core API Reference

gsap.to() - Animate TO specified values

```javascript
gsap.to(targets, {
  // Transform properties (GPU-accelerated)
  x: 100,              // translateX in pixels
  y: 50,               // translateY in pixels
  rotation: 360,       // degrees
  scale: 1.5,          // uniform scale
  scaleX: 1.2,
  scaleY: 0.8,

  // Opacity
  opacity: 0.5,
  autoAlpha: 0,        // opacity + visibility:hidden at 0

  // Special properties
  duration: 1,         // seconds (default: 0.5)
  delay: 0.5,          // delay before start
  ease: "power2.out",  // easing function
  repeat: -1,          // -1 = infinite
  yoyo: true,          // alternate direction
  stagger: 0.1,        // time between each element

  // Callbacks
  onStart: () => {},
  onUpdate: () => {},
  onComplete: () => {},
  onRepeat: () => {}
});
```

gsap.from() - Animate FROM specified values

```javascript
gsap.from(".element", {
  opacity: 0,
  y: 100,
  scale: 0.5,
  duration: 1,
  ease: "back.out(1.7)"  // Great for entrance animations
});
```

gsap.fromTo() - Define both start AND end

```javascript
gsap.fromTo(".element",
  { opacity: 0, x: -100 },       // FROM
  { opacity: 1, x: 100, duration: 1 }  // TO (special props go here)
);
```

gsap.set() - Instant property setting

```javascript
gsap.set(".element", {
  x: 100,
  transformOrigin: "center center",
  visibility: "visible"
});
```

1.2 Easing Functions Guide

Standard Eases (use `.out` for 95% of UI)

Ease	Intensity	Best For
`power1.out`	Subtle	Micro-interactions
`power2.out`	Moderate	**Default UI animations**
`power3.out`	Strong	Dramatic reveals
`power4.out`	Very dramatic	Hero sections

Ease	Intensity	Best For
`expo.out`	Exponential	High-impact moments

Special Eases

```javascript
// Back - overshoot effect (great for modals/buttons)
ease: "back.out(1.7)"     // 1.7 is default overshoot amount

// Elastic - spring oscillation
ease: "elastic.out(1, 0.3)"  // (amplitude, period)

// Bounce - ball bounce
ease: "bounce.out"

// Steps - discrete frames
ease: "steps(5)"
```

Ease Directions

- `.in` - Starts slow, accelerates (entering screen)
- `.out` - Starts fast, decelerates (UI default)
- `.inOut` - Smooth both ends (looping, symmetrical) `Gsapify`

1.3 Timeline System

Creating and Chaining

```javascript
```

```javascript
const tl = gsap.timeline({
  defaults: { duration: 1, ease: "power2.out" },
  paused: true,    // start paused
  repeat: -1,      // infinite loop
  yoyo: true       // reverse on repeat
});

tl.to(".box1", { x: 100 })
  .to(".box2", { y: 50 }, "-=0.5")      // overlap 0.5s
  .to(".box3", { rotation: 90 }, "<")    // start with previous
  .addLabel("spin")
  .to(".box4", { scale: 2 }, "spin")     // at label
  .to(".box5", { opacity: 0 }, "spin+=0.25");  // 0.25s after label
```

Position Parameter Reference

Syntax	Description
(none)	After previous animation
`3`	Absolute 3 seconds from start
`"+=1"`	1s gap after previous
`"-=0.5"`	Overlap by 0.5s
`"<"`	Start of previous animation
`">"`	End of previous animation
`"<0.5"`	0.5s after start of previous
`"labelName"`	At label position
`"labelName+=0.5"`	0.5s after label

Timeline Control

```javascript
tl.play();       // Play forward
tl.pause();      // Pause
tl.reverse();    // Play backwards
tl.restart();    // Go to start and play
tl.progress(0.5);  // Jump to 50%
tl.timeScale(2);   // 2x speed
```

Nesting Timelines (Modular Architecture)

javascript

```
function createIntro() {
  const tl = gsap.timeline();
  tl.from(".logo", { scale: 0, duration: 1 })
    .from(".tagline", { opacity: 0, y: 20 });
  return tl;
}

const master = gsap.timeline();
master
  .add(createIntro())
  .add(createMain(), "-=0.5")
  .add(createOutro());
```

1.4 Stagger Animations

Basic Stagger

javascript

```
gsap.to(".card", {
  y: 100,
  opacity: 1,
  stagger: 0.1  // 0.1s between each
});
```

Advanced Stagger Object

javascript

```
gsap.to(".card", {
  y: 100,
  stagger: {
    each: 0.1,          // Time between starts
    from: "center",     // "start", "end", "center", "edges", "random", or index
    grid: [4, 5],       // For grid layouts [rows, cols]
    axis: "y",          // "x", "y", or null (both)
    ease: "power2.in"   // Distribution ease
  }
});
```

Grid Stagger (Perfect for card grids)

javascript

```
gsap.from(".grid-item", {
  opacity: 0,
  scale: 0,
  stagger: {
    grid: "auto",       // Auto-detect grid
    from: "center",     // Emanate from center
    amount: 1.5,        // Total stagger time
    ease: "power2.out"
  }
});
```

1.5 3D Transforms with GSAP

Core 3D Properties

javascript

```
gsap.to(element, {
  rotationX: 45,        // Rotate around X-axis (degrees)
  rotationY: 180,       // Rotate around Y-axis
  rotationZ: 90,        // Same as 'rotation'
  z: -300,              // Translate along Z-axis
  transformPerspective: 500,  // Per-element perspective
  transformOrigin: "50% 50% -400px"  // 3D origin (x y z)
});
```

3D Card Flip Implementation

javascript

```
// HTML: .cardWrapper > .card > .front + .back
// CSS: .back { transform: rotateY(-180deg); }

// GSAP Setup
gsap.set(".cardWrapper", { perspective: 800 });
gsap.set(".card", { transformStyle: "preserve-3d" });
gsap.set(".back", { rotationY: -180 });
gsap.set([".front", ".back"], { backfaceVisibility: "hidden" });

// Flip Timeline
const flipTL = gsap.timeline({ paused: true });
flipTL.to(".card", {
  rotationY: -180,
  duration: 0.8,
  ease: "power2.inOut"
});

// Controls
const flipCard = () => flipTL.play();
const unflipCard = () => flipTL.reverse();
```

Card Flip with Lift Effect

javascript

```
const flipTL = gsap.timeline({ paused: true });
flipTL
  .to(".card", { z: 50, duration: 0.4 })
  .to(".card", { rotationY: 180, duration: 0.6, ease: "power2.inOut" }, 0)
  .to(".card", { z: 0, duration: 0.4 }, 0.4);
```

1.6 ScrollTrigger Plugin

Basic Setup

javascript

```javascript
import { ScrollTrigger } from "gsap/ScrollTrigger";
gsap.registerPlugin(ScrollTrigger);

gsap.to(".element", {
  x: 500,
  scrollTrigger: {
    trigger: ".element",
    start: "top center",     // "trigger viewport"
    end: "bottom top",
    scrub: 1,               // Smooth scrub (seconds)
    pin: true,              // Pin during animation
    markers: true           // Debug markers
  }
});
```

Start/End Position Syntax

```javascript
start: "top center"       // Top of trigger hits center of viewport
start: "top 80%"          // 80% from viewport top
start: "top bottom-=100px" // 100px above viewport bottom
start: () => `+=${elem.offsetHeight}`  // Dynamic calculation
```

Pin Configuration

```javascript
scrollTrigger: {
  pin: true,              // Pin the trigger element
  pinSpacing: true,       // Add padding (default)
  pinSpacing: false,      // No spacing (flex/absolute)
  anticipatePin: 1,       // Prevents flash on fast scroll
}
```

Scrub Options

```javascript
scrub: true    // Direct link to scrollbar
scrub: 0.5     // 0.5s smoothing
scrub: 1       // 1s smoothing (buttery)
scrub: 3       // Very smooth, laggy feel
```

All Callbacks

javascript

```
ScrollTrigger.create({
  trigger: ".element",
  onEnter: (self) => {},       // Forward into trigger
  onLeave: (self) => {},       // Forward past end
  onEnterBack: (self) => {},   // Backward past end
  onLeaveBack: (self) => {},   // Backward past start
  onUpdate: (self) => {
    console.log(self.progress); // 0-1 progress
    console.log(self.direction); // 1 or -1
  },
  onToggle: (self) => {},      // When active changes
  onRefresh: (self) => {}      // On resize/refresh
});
```

Snap Configuration

javascript

```
snap: 0.1                    // Snap to 10% increments
snap: [0, 0.25, 0.5, 0.75, 1]   // Specific values
snap: "labels"               // Timeline labels
snap: {
  snapTo: "labels",
  duration: { min: 0.2, max: 3 },
  delay: 0.2,
  ease: "power1.inOut"
}
```

Horizontal Scroll

javascript

```javascript
let sections = gsap.utils.toArray(".panel");

gsap.to(sections, {
  xPercent: -100 * (sections.length - 1),
  ease: "none",
  scrollTrigger: {
    trigger: ".container",
    pin: true,
    scrub: 1,
    snap: 1 / (sections.length - 1),
    end: () => "+=" + document.querySelector(".container").offsetWidth
  }
});
```

Batch Processing

javascript

```javascript
ScrollTrigger.batch(".box", {
  interval: 0.1,
  batchMax: 3,
  onEnter: (batch) => gsap.to(batch, {
    opacity: 1,
    y: 0,
    stagger: 0.15
  }),
  onLeave: (batch) => gsap.set(batch, { opacity: 0, y: -100 })
});
```

1.7 SplitText Plugin

Basic Usage

javascript

```
import { SplitText } from "gsap/SplitText";
gsap.registerPlugin(SplitText);

let split = SplitText.create(".headline", {
  type: "lines, words, chars"
});

gsap.from(split.chars, {
  y: 50,
  opacity: 0,
  stagger: 0.02,
  ease: "back.out(1.7)"
});
```

With Mask (Line Reveals)

javascript

```
SplitText.create(".paragraph", {
  type: "lines",
  mask: "lines",       // Wraps in clip elements
  autoSplit: true,     // Re-splits on resize
  onSplit: (self) => {
    return gsap.from(self.lines, {
      yPercent: 100,
      stagger: 0.1,
      ease: "power4.out"
    });
  }
});
```

Revert After Animation

javascript

```
let split = SplitText.create(".text", { type: "words" });

gsap.from(split.words, {
  y: 50,
  opacity: 0,
  stagger: 0.05,
  onComplete: () => split.revert()  // Restore original HTML
});
```

1.8 FLIP Plugin

Basic FLIP Pattern

javascript

```
import { Flip } from "gsap/Flip";
gsap.registerPlugin(Flip);

// 1. Capture state
const state = Flip.getState(".item");

// 2. Make DOM changes
container.appendChild(item);

// 3. Animate transition
Flip.from(state, {
  duration: 0.6,
  ease: "power2.inOut",
  scale: true,          // Use scale instead of width/height
  absolute: true        // Position absolute during animation
});
```

Filter/Sort Animation

javascript

```
function updateFilter(filter) {
  const state = Flip.getState(".item");

  items.forEach(item => {
    item.classList.toggle("hidden", !matchesFilter(item, filter));
  });

  Flip.from(state, {
    duration: 0.7,
    scale: true,
    absolute: true,
    onEnter: elements => gsap.fromTo(elements,
      { opacity: 0, scale: 0 },
      { opacity: 1, scale: 1 }
    ),
    onLeave: elements => gsap.to(elements,
      { opacity: 0, scale: 0 }
    )
  });
}
```

1.9 Performance Optimization

force3D Options

javascript

```
gsap.config({ force3D: "auto" });  // Default: GPU during animation only

gsap.to(elem, { x: 100, force3D: true });   // Always GPU
gsap.to(elem, { scale: 3, force3D: false }); // Fix Safari/Chrome blur
```

quickTo / quickSetter (High-Frequency Updates)

javascript

```
// quickTo - animated with built-in tween
const xTo = gsap.quickTo("#cursor", "x", { duration: 0.8, ease: "power3" });
const yTo = gsap.quickTo("#cursor", "y", { duration: 0.8, ease: "power3" });

window.addEventListener("mousemove", (e) => {
  xTo(e.clientX);
  yTo(e.clientY);
});

// quickSetter - instant setting (50-250% faster than gsap.set)
const xSetter = gsap.quickSetter("#elem", "x", "px");
const ySetter = gsap.quickSetter("#elem", "y", "px");
```

Performance Best Practices

1. **Animate transforms** (x, y, rotation, scale) - GPU accelerated
2. **Avoid layout properties** (left, top, width, height) - causes reflow
3. **Use autoAlpha** instead of opacity for fade-outs
4. **Use stagger** for multiple elements - prevents frame drops
5. **Apply will-change sparingly** - only to actively animating elements

---

1.10 React Integration

useGSAP Hook (Official)

javascript

```
import { useRef } from 'react';
import gsap from 'gsap';
import { useGSAP } from '@gsap/react';

gsap.registerPlugin(useGSAP);

function AnimatedCard() {
  const container = useRef();

  useGSAP(() => {
    gsap.to('.box', { x: 360 });
  }, { scope: container }); // Auto cleanup on unmount

  return (
    <div ref={container}>
      <div className="box">Animated</div>
    </div>
  );
}
```

Context-Safe Event Handlers

javascript

```
function CardWithClick() {
  const container = useRef();

  const { contextSafe } = useGSAP({ scope: container });

  const handleClick = contextSafe(() => {
    gsap.to('.card', { rotationY: 180 });
  });

  return (
    <div ref={container}>
      <button onClick={handleClick} className="card">Flip</button>
    </div>
  );
}
```

1.11 Accessibility

Reduced Motion Support

javascript

```
const mm = gsap.matchMedia();

mm.add("(prefers-reduced-motion: no-preference)", () => {
  gsap.from(".box", {
    rotation: 360,
    y: 100,
    ease: "back.out"
  });
});

mm.add("(prefers-reduced-motion: reduce)", () => {
  gsap.from(".box", { opacity: 0 }); // Simple fade only
});
```

PART 2: THREE.JS 3D LIBRARY

2.1 Core Setup Boilerplate

javascript

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

// Scene
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x1a1a2e);

// Camera
const camera = new THREE.PerspectiveCamera(
  75,                          // FOV (45-75 typical)
  window.innerWidth / window.innerHeight,  // Aspect ratio
  0.1,                         // Near plane
  1000                          // Far plane
);
camera.position.set(0, 2, 5);

// Renderer
const renderer = new THREE.WebGLRenderer({
  antialias: true,
  alpha: true
});
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap;
renderer.toneMapping = THREE.ACESFilmicToneMapping;
document.body.appendChild(renderer.domElement);

// Controls
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
controls.dampingFactor = 0.05;

// Lights
const ambientLight = new THREE.AmbientLight(0xffffff, 0.4);
scene.add(ambientLight);

const directionalLight = new THREE.DirectionalLight(0xffffff, 1);
directionalLight.position.set(5, 10, 5);
directionalLight.castShadow = true;
scene.add(directionalLight);

// Resize Handler
window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
```

```
  renderer.setSize(window.innerWidth, window.innerHeight);
});

// Animation Loop
function animate() {
  controls.update();
  renderer.render(scene, camera);
}
renderer.setAnimationLoop(animate);
```

2.2 Geometry Reference

javascript

```
// PlaneGeometry - IDEAL FOR CARDS
new THREE.PlaneGeometry(width, height, widthSegments, heightSegments);

// BoxGeometry - For cards with thickness
new THREE.BoxGeometry(width, height, depth);

// SphereGeometry
new THREE.SphereGeometry(radius, widthSegments, heightSegments);

// Custom BufferGeometry
const geometry = new THREE.BufferGeometry();
const vertices = new Float32Array([...]);
geometry.setAttribute('position', new THREE.BufferAttribute(vertices, 3));
geometry.computeVertexNormals();
```

2.3 Material Reference

MeshStandardMaterial (PBR - Recommended)

javascript

```javascript
const material = new THREE.MeshStandardMaterial({
  color: 0xffffff,
  map: colorTexture,

  // PBR
  metalness: 0.5,
  roughness: 0.5,

  // Maps
  normalMap: normalTexture,
  aoMap: aoTexture,
  envMap: envMapTexture,

  // Transparency
  transparent: true,
  opacity: 1.0,

  // CRITICAL FOR CARDS
  side: THREE.DoubleSide
});
```

MeshPhysicalMaterial (Advanced PBR)

javascript

```javascript
const material = new THREE.MeshPhysicalMaterial({
  // Glass/transmission
  transmission: 1.0,
  thickness: 0.5,
  ior: 1.5,

  // Clearcoat
  clearcoat: 1.0,
  clearcoatRoughness: 0.0,

  // Iridescence
  iridescence: 0.5,
  iridescenceIOR: 1.3
});
```

Material Array for Different Front/Back

javascript

```
// BoxGeometry faces: [right, left, top, bottom, front, back]
const materials = [
  sideMaterial,  // right
  sideMaterial,  // left
  sideMaterial,  // top
  sideMaterial,  // bottom
  frontMaterial, // front face
  backMaterial   // back face
];
const card = new THREE.Mesh(geometry, materials);
```

2.4 3D Card Implementation

Two-Plane Card (Different Front/Back)

javascript

```
function createCard(frontTexture, backTexture) {
  const group = new THREE.Group();
  const geometry = new THREE.PlaneGeometry(2, 3);

  // Front
  const frontMesh = new THREE.Mesh(geometry,
    new THREE.MeshStandardMaterial({ map: frontTexture })
  );

  // Back (rotated 180°)
  const backMesh = new THREE.Mesh(geometry,
    new THREE.MeshStandardMaterial({ map: backTexture })
  );
  backMesh.rotation.y = Math.PI;

  group.add(frontMesh, backMesh);
  return group;
}
```

Card Flip with GSAP

javascript

```javascript
function flipCard(card) {
  gsap.to(card.rotation, {
    y: card.rotation.y + Math.PI,
    duration: 0.6,
    ease: "power2.inOut"
  });
}

// With lift effect
function flipCardWithLift(card) {
  const tl = gsap.timeline();
  tl.to(card.position, { y: "+=0.5", duration: 0.3, ease: "power2.out" })
    .to(card.rotation, { y: Math.PI, duration: 0.6, ease: "power2.inOut" }, "<")
    .to(card.position, { y: "-=0.5", duration: 0.3, ease: "power2.in" }, "-=0.3");
  return tl;
}
```

Card Grid Layout

javascript

```javascript
function createCardGrid(rows, cols, spacing = 2.5) {
  const cards = [];
  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      const card = createCard(frontTex, backTex);
      card.position.set(
        col * spacing - (cols - 1) * spacing / 2,
        row * spacing - (rows - 1) * spacing / 2,
        0
      );
      scene.add(card);
      cards.push(card);
    }
  }
  return cards;
}
```

Interactive Card Selection (Raycasting)

javascript

```javascript
const raycaster = new THREE.Raycaster();
const mouse = new THREE.Vector2();

function onMouseClick(event) {
  mouse.x = (event.clientX / window.innerWidth) * 2 - 1;
  mouse.y = -(event.clientY / window.innerHeight) * 2 + 1;

  raycaster.setFromCamera(mouse, camera);
  const intersects = raycaster.intersectObjects(cards, true);

  if (intersects.length > 0) {
    flipCard(intersects[0].object.parent);
  }
}
```

2.5 Post-Processing

EffectComposer Setup

javascript

```javascript
import { EffectComposer } from 'three/addons/postprocessing/EffectComposer.js';
import { RenderPass } from 'three/addons/postprocessing/RenderPass.js';
import { UnrealBloomPass } from 'three/addons/postprocessing/UnrealBloomPass.js';
import { OutputPass } from 'three/addons/postprocessing/OutputPass.js';

const composer = new EffectComposer(renderer);
composer.addPass(new RenderPass(scene, camera));

// Bloom
const bloomPass = new UnrealBloomPass(
  new THREE.Vector2(window.innerWidth, window.innerHeight),
  1.5,  // strength
  0.4,  // radius
  0.85  // threshold
);
composer.addPass(bloomPass);

// Always last for sRGB conversion
composer.addPass(new OutputPass());

// In animation loop:
composer.render(); // Instead of renderer.render()
```

2.6 GSAP + Three.js Integration

Animating Mesh Properties

javascript

```javascript
// Position
gsap.to(mesh.position, { x: 5, y: 2, duration: 2, ease: "power2.inOut" });

// Rotation
gsap.to(mesh.rotation, { y: Math.PI * 2, duration: 3, repeat: -1, ease: "none" });

// Scale
gsap.to(mesh.scale, { x: 1.5, y: 1.5, z: 1.5, duration: 1, yoyo: true, repeat: -1 });

// Material opacity
mesh.material.transparent = true;
gsap.to(mesh.material, { opacity: 0, duration: 1 });

// Color
const targetColor = new THREE.Color(0xff0000);
gsap.to(mesh.material.color, { r: targetColor.r, g: targetColor.g, b: targetColor.b });
```

ScrollTrigger with Three.js

javascript

```javascript
gsap.to(model.rotation, {
  y: Math.PI * 2,
  scrollTrigger: {
    trigger: ".scroll-section",
    start: "top top",
    end: "bottom bottom",
    scrub: 1,
    onUpdate: (self) => {
      model.position.y = self.progress * 5;
    }
  }
});
```

2.7 React-Three-Fiber

Basic Setup

jsx

```jsx
import { Canvas } from '@react-three/fiber';
import { OrbitControls, Environment } from '@react-three/drei';

function App() {
  return (
    <Canvas camera={{ position: [0, 0, 5], fov: 75 }}>
      <ambientLight intensity={0.5} />
      <pointLight position={[10, 10, 10]} />
      <OrbitControls enableDamping />
      <Environment preset="sunset" />
      <MyScene />
    </Canvas>
  );
}
```

useFrame Hook

jsx

```jsx
import { useFrame } from '@react-three/fiber';
import { useRef } from 'react';

function RotatingBox() {
  const meshRef = useRef();

  useFrame((state, delta) => {
    meshRef.current.rotation.x += delta;
    meshRef.current.rotation.y += delta * 0.5;
  });

  return (
    <mesh ref={meshRef}>
      <boxGeometry args={[1, 1, 1]} />
      <meshStandardMaterial color="orange" />
    </mesh>
  );
}
```

Post-Processing in R3F

```jsx
import { EffectComposer, Bloom, Vignette } from '@react-three/postprocessing';

function Effects() {
  return (
    <EffectComposer>
      <Bloom luminanceThreshold={0.9} intensity={1.5} />
      <Vignette offset={0.1} darkness={1.1} />
    </EffectComposer>
  );
}
```

2.8 Performance Optimization

Instanced Rendering (Many Objects)

```javascript
const instancedMesh = new THREE.InstancedMesh(geometry, material, 10000);

const dummy = new THREE.Object3D();
for (let i = 0; i < 10000; i++) {
  dummy.position.set(Math.random() * 100, Math.random() * 100, Math.random() * 100);
  dummy.updateMatrix();
  instancedMesh.setMatrixAt(i, dummy.matrix);
}
instancedMesh.instanceMatrix.needsUpdate = true;
```

Proper Disposal

```javascript
```

```
function disposeObject(object) {
  object.traverse((child) => {
    if (child instanceof THREE.Mesh) {
      child.geometry.dispose();
      if (Array.isArray(child.material)) {
        child.material.forEach(mat => mat.dispose());
      } else {
        child.material.dispose();
      }
    }
  });
}
```

PART 3: WEBGL SHADERS

3.1 GLSL Fundamentals

Data Types

Type	Description	Example
`float`	Single number	`float x = 1.0;`
`vec2`	2D vector	`vec2 uv = vec2(0.5, 0.5);`
`vec3`	3D vector (xyz/rgb)	`vec3 color = vec3(1.0, 0.0, 0.0);`
`vec4`	4D vector (rgba)	`vec4 pos = vec4(1.0, 0.0, 0.0, 1.0);`
`mat4`	4x4 matrix	`mat4 model = mat4(1.0);`
`sampler2D`	Texture	`uniform sampler2D u_tex;`

Essential Functions

```
glsl
```

```
mix(a, b, t)          // Linear interpolation
smoothstep(e0, e1, x)  // Smooth 0-1 transition
step(edge, x)         // 0 if x < edge, else 1
clamp(x, min, max)     // Constrain value
fract(x)              // Fractional part
abs(x)                // Absolute value
length(v)              // Vector length
normalize(v)          // Unit vector
dot(a, b)             // Dot product
texture2D(sampler, uv) // Sample texture
```

3.2 Three.js ShaderMaterial Template

javascript

```
const material = new THREE.ShaderMaterial({
  uniforms: {
    u_time: { value: 0 },
    u_resolution: { value: new THREE.Vector2(window.innerWidth, window.innerHeight) },
    u_mouse: { value: new THREE.Vector2(0.5, 0.5) },
    u_color: { value: new THREE.Color(0x00ffff) },
    u_texture: { value: null }
  },
  vertexShader: `
    varying vec2 vUv;
    varying vec3 vPosition;

    void main() {
      vUv = uv;
      vPosition = position;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
  `,
  fragmentShader: `
    precision mediump float;

    uniform float u_time;
    uniform vec2 u_resolution;
    uniform vec3 u_color;

    varying vec2 vUv;

    void main() {
      vec3 color = u_color * (0.5 + 0.5 * sin(u_time));
      gl_FragColor = vec4(color, 1.0);
    }
  `,
  transparent: true,
  side: THREE.DoubleSide
});

// Update uniforms in animation loop
function animate() {
  material.uniforms.u_time.value = clock.getElapsedTime();
  requestAnimationFrame(animate);
}
```

3.3 Noise Functions

Simplex Noise 2D

glsl

```
vec3 permute(vec3 x) { return mod(((x*34.0)+1.0)*x, 289.0); }

float snoise(vec2 v) {
  const vec4 C = vec4(0.211324865405187, 0.366025403784439,
                      -0.577350269189626, 0.024390243902439);
  vec2 i  = floor(v + dot(v, C.yy));
  vec2 x0 = v - i + dot(i, C.xx);
  vec2 i1 = (x0.x > x0.y) ? vec2(1.0, 0.0) : vec2(0.0, 1.0);
  vec4 x12 = x0.xyxy + C.xxzz;
  x12.xy -= i1;
  i = mod(i, 289.0);
  vec3 p = permute(permute(i.y + vec3(0.0, i1.y, 1.0)) + i.x + vec3(0.0, i1.x, 1.0));
  vec3 m = max(0.5 - vec3(dot(x0,x0), dot(x12.xy,x12.xy), dot(x12.zw,x12.zw)), 0.0);
  m = m*m; m = m*m;
  vec3 x = 2.0 * fract(p * C.www) - 1.0;
  vec3 h = abs(x) - 0.5;
  vec3 ox = floor(x + 0.5);
  vec3 a0 = x - ox;
  m *= 1.79284291400159 - 0.85373472095314 * (a0*a0 + h*h);
  vec3 g;
  g.x = a0.x * x0.x + h.x * x0.y;
  g.yz = a0.yz * x12.xz + h.yz * x12.yw;
  return 130.0 * dot(m, g);
}
```

Fractal Brownian Motion (fBm)

glsl

```glsl
#define NUM_OCTAVES 5

float fbm(vec2 x) {
  float v = 0.0;
  float a = 0.5;
  vec2 shift = vec2(100.0);
  mat2 rot = mat2(cos(0.5), sin(0.5), -sin(0.5), cos(0.5));

  for (int i = 0; i < NUM_OCTAVES; ++i) {
    v += a * snoise(x);
    x = rot * x * 2.0 + shift;
    a *= 0.5;
  }
  return v;
}
```

3.4 Gradient Mesh (Apple-style)

Cosine Palette

glsl

```glsl
vec3 palette(float t, vec3 a, vec3 b, vec3 c, vec3 d) {
  return a + b * cos(6.28318 * (c * t + d));
}

void main() {
  float t = length(vUv - 0.5) + u_time * 0.1;
  vec3 color = palette(t,
    vec3(0.5, 0.5, 0.5),    // brightness
    vec3(0.5, 0.5, 0.5),    // contrast
    vec3(1.0, 1.0, 1.0),    // frequency
    vec3(0.0, 0.33, 0.67)   // phase offset
  );
  gl_FragColor = vec4(color, 1.0);
}
```

Animated Gradient with Noise

glsl

```glsl
uniform float u_time;
varying vec2 vUv;

void main() {
  vec2 uv = vUv;
  float noise = snoise(vec3(uv * 2.0, u_time * 0.2)) * 0.1;
  uv += noise;

  vec3 color1 = vec3(0.2, 0.4, 0.8);  // Blue
  vec3 color2 = vec3(0.8, 0.2, 0.6);  // Purple
  vec3 color3 = vec3(0.9, 0.5, 0.3);  // Orange

  float gradient = smoothstep(0.0, 0.5, uv.x + noise);
  vec3 color = mix(color1, color2, gradient);
  color = mix(color, color3, smoothstep(0.5, 1.0, uv.y + noise));

  gl_FragColor = vec4(color, 1.0);
}
```

3.5 Metaball/Blob Effects

Smooth Union SDF

glsl

```
float sdCircle(vec2 p, float r) {
  return length(p) - r;
}

float smin(float a, float b, float k) {
  float h = max(k - abs(a - b), 0.0) / k;
  return min(a, b) - h * h * k * 0.25;
}

void main() {
  vec2 uv = (gl_FragCoord.xy * 2.0 - u_resolution) / min(u_resolution.x, u_resolution.y);

  // Animated blob positions
  vec2 blob1 = vec2(sin(u_time * 0.7) * 0.5, cos(u_time * 0.5) * 0.3);
  vec2 blob2 = vec2(cos(u_time * 0.6) * 0.4, sin(u_time * 0.8) * 0.4);
  vec2 blob3 = vec2(sin(u_time * 0.4) * 0.3, cos(u_time * 0.9) * 0.5);

  float d1 = sdCircle(uv - blob1, 0.3);
  float d2 = sdCircle(uv - blob2, 0.25);
  float d3 = sdCircle(uv - blob3, 0.2);

  float d = smin(smin(d1, d2, 0.3), d3, 0.3);

  vec3 color = vec3(0.0);
  if (d < 0.0) {
    color = mix(vec3(0.2, 0.5, 0.9), vec3(0.9, 0.3, 0.6), -d * 3.0);
  } else {
    color = vec3(0.1, 0.2, 0.4) * (1.0 / (d * 10.0 + 1.0));
  }

  gl_FragColor = vec4(color, 1.0);
}
```

3.6 Wave Distortion

Vertex Displacement

glsl

```
// Vertex Shader
uniform float u_time;
uniform float u_frequency;
uniform float u_amplitude;

varying vec2 vUv;

void main() {
  vUv = uv;
  vec3 pos = position;

  pos.z += sin(pos.x * u_frequency + u_time) * u_amplitude;
  pos.z += sin(pos.y * u_frequency * 0.5 + u_time * 0.7) * u_amplitude * 0.5;

  gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}
```

Ripple Effect

glsl

```
uniform float u_time;
uniform vec2 u_mouse;
varying vec2 vUv;

void main() {
  vec2 uv = vUv;
  float dist = distance(uv, u_mouse);

  float ripple = sin(dist * 30.0 - u_time * 5.0) * 0.02;
  ripple *= smoothstep(0.5, 0.0, dist);

  vec2 distortedUV = uv + ripple * normalize(uv - u_mouse);
  gl_FragColor = texture2D(u_texture, distortedUV);
}
```

3.7 Glass/Frosted Effects

Fresnel Effect

glsl

```glsl
float fresnel(vec3 viewDir, vec3 normal, float power) {
  return pow(1.0 + dot(viewDir, normal), power);
}
```

Chromatic Aberration

glsl

```glsl
uniform float iorR, iorG, iorB;  // Different IOR per channel

void main() {
  vec3 refractR = refract(viewDir, normal, 1.0 / iorR);
  vec3 refractG = refract(viewDir, normal, 1.0 / iorG);
  vec3 refractB = refract(viewDir, normal, 1.0 / iorB);

  float r = textureCube(envMap, refractR).r;
  float g = textureCube(envMap, refractG).g;
  float b = textureCube(envMap, refractB).b;

  gl_FragColor = vec4(r, g, b, 1.0);
}
```

3.8 Animating Shaders with GSAP

Direct Uniform Animation

javascript

```javascript
gsap.to(material.uniforms.u_progress, {
  value: 1.0,
  duration: 2,
  ease: "power2.inOut"
});
```

Timeline Control

javascript

```javascript
const tl = gsap.timeline({ paused: true });
tl.to(material.uniforms.u_progress, { value: 1, duration: 1 })
  .to(material.uniforms.u_distortion, { value: 0.5, duration: 0.8 }, "-=0.5")
  .to(material.uniforms.u_color.value, { r: 1, g: 0, b: 0.5, duration: 0.5 });
```

ScrollTrigger Integration

```javascript
ScrollTrigger.create({
  trigger: ".section",
  start: "top center",
  end: "bottom center",
  scrub: 1,
  onUpdate: (self) => {
    material.uniforms.u_progress.value = self.progress;
    material.uniforms.u_strength.value = gsap.utils.interpolate(0, 1, self.progress);
  }
});
```

3.9 Shadertoy Conversion

Uniform Mapping

Shadertoy	Three.js
`iTime`	`uniform float u_time`
`iResolution`	`uniform vec3 u_resolution`
`fragCoord`	`gl_FragCoord.xy`
`fragColor`	`gl_FragColor`
`iMouse`	`uniform vec4 u_mouse`

Conversion Template

```glsl
```

```
uniform vec3 u_resolution;
uniform float u_time;
uniform vec4 u_mouse;

// Paste Shadertoy mainImage() here
void mainImage(out vec4 fragColor, in vec2 fragCoord) {
  // Original code
}

void main() {
  mainImage(gl_FragColor, gl_FragCoord.xy);
}
```

3.10 Performance Best Practices

1. **Move calculations to vertex shader** when possible (runs per-vertex vs per-pixel)
2. **Avoid conditionals** - use `mix()` and `step()` instead
3. **Minimize texture samples** - combine into single read when possible
4. **Use `mediump` precision** for mobile compatibility
5. **Pack data** into single textures using RGBA channels
6. **Test on mobile** - limit to 4-8 texture samples per shader

---

PART 4: INTEGRATION PATTERNS

4.1 Complete 3D Card Flip (GSAP + Three.js)

```javascript
```

```
function setupCardFlip(cardGroup, camera) {
  // Initial setup
  gsap.set(cardGroup.rotation, { y: 0 });

  // Flip timeline
  const flipTL = gsap.timeline({ paused: true });
  flipTL
    .to(cardGroup.position, { z: 1, duration: 0.3, ease: "power2.out" })
    .to(cardGroup.rotation, { y: Math.PI, duration: 0.6, ease: "power2.inOut" }, 0)
    .to(cardGroup.position, { z: 0, duration: 0.3, ease: "power2.in" }, 0.3);

  return {
    flip: () => flipTL.play(),
    unflip: () => flipTL.reverse(),
    isFlipped: () => flipTL.progress() > 0.5
  };
}
```

4.2 Scroll-Animated Shader Background

javascript

```
// Shader material
const bgMaterial = new THREE.ShaderMaterial({
  uniforms: {
    u_time: { value: 0 },
    u_scroll: { value: 0 },
    u_color1: { value: new THREE.Color(0x1a1a2e) },
    u_color2: { value: new THREE.Color(0x16213e) }
  },
  fragmentShader: gradientShader
});

// GSAP ScrollTrigger integration
ScrollTrigger.create({
  trigger: "body",
  start: "top top",
  end: "bottom bottom",
  onUpdate: (self) => {
    bgMaterial.uniforms.u_scroll.value = self.progress;
  }
});

// Animation loop
function animate() {
  bgMaterial.uniforms.u_time.value = clock.getElapsedTime();
  renderer.render(scene, camera);
}
```

4.3 React Integration (Complete Example)

```
jsx
```

```
import { Canvas, useFrame } from '@react-three/fiber';
import { useRef, useEffect } from 'react';
import gsap from 'gsap';

function AnimatedCard({ onClick }) {
  const meshRef = useRef();

  useEffect(() => {
    // GSAP entrance animation
    gsap.from(meshRef.current.scale, {
      x: 0, y: 0, z: 0,
      duration: 1,
      ease: "back.out(1.7)"
    });
  }, []);

  const handleClick = () => {
    gsap.to(meshRef.current.rotation, {
      y: meshRef.current.rotation.y + Math.PI,
      duration: 0.6,
      ease: "power2.inOut"
    });
    onClick?.();
  };

  return (
    <mesh ref={meshRef} onClick={handleClick}>
      <planeGeometry args={[2, 3]} />
      <meshStandardMaterial color="white" side={THREE.DoubleSide} />
    </mesh>
  );
}

function App() {
  return (
    <Canvas camera={{ position: [0, 0, 5] }}>
      <ambientLight intensity={0.5} />
      <AnimatedCard />
    </Canvas>
  );
}
```

QUICK REFERENCE CHEATSHEET

GSAP Essentials

```javascript
gsap.to(".el", { x: 100, duration: 1, ease: "power2.out" });
gsap.from(".el", { opacity: 0, y: 50 });
gsap.timeline().to().to().to();  // Chaining
```

Three.js Essentials

```javascript
new THREE.Scene();
new THREE.PerspectiveCamera(75, aspect, 0.1, 1000);
new THREE.WebGLRenderer({ antialias: true });
new THREE.Mesh(geometry, material);
```

Shader Essentials

```glsl
uniform float u_time;
varying vec2 vUv;
gl_FragColor = vec4(color, 1.0);
mix(a, b, t);  smoothstep(0.0, 1.0, x);
```

Key URLs

- GSAP Docs: gsap.com/docs/v3
- Three.js Docs: threejs.org/docs
- Book of Shaders: thebookofshaders.com
- Shadertoy: shadertoy.com
- LYGIA Shader Library: lygia.xyz

Note: As of 2024, Webflow acquired GSAP making ALL plugins (ScrollTrigger, SplitText, FLIP, etc.) completely FREE for commercial use.